

ENTREGABLE PDF

Backend IoT histórico con Supabase y Wokwi

Universidad Técnica Estatal de Quevedo

Objetivo: implementar un sistema donde un ESP32 simulado en Wokwi obtiene temperatura y humedad mediante DHT22, envía los datos a una API REST y los almacena en Supabase para consultas históricas.

1. Arquitectura del sistema

Etapas	Componente	Función
1	Wokwi + ESP32 + DHT22	Captura temperatura y humedad.
2	WiFi + HTTP	Envía datos en JSON.
3	API REST Flask	Valida y registra mediciones.
4	Supabase	Almacena el histórico.
5	GET /mediciones	Consulta los registros.

2. Base de datos Supabase

La tabla **dispositivos** identifica el equipo y **mediciones** conserva cada variable con su valor, unidad y timestamp.

```
create table if not exists public.dispositivos (  
  device_id text primary key,  
  nombre text not null,  
  ubicacion text,  
  creado_en timestampz default now()  
);  
  
create table if not exists public.mediciones (  
  id bigint generated always as identity primary key,  
  device_id text not null references public.dispositivos(device_id),  
  tipo_variable text not null,  
  valor numeric not null,  
  unidad text not null,  
  timestamp timestampz not null default now()  
);  
  
insert into public.dispositivos (device_id, nombre, ubicacion)  
values ('sensor_001', 'ESP32 DHT22', 'Quevedo, Ecuador')  
on conflict (device_id) do nothing;  
  
select * from public.mediciones  
order by timestamp desc  
limit 50;
```

3. Backend API REST

El backend recibe POST desde el ESP32 y guarda los datos en Supabase. También permite consultar el histórico mediante GET.

```
from flask import Flask, request, jsonify
import os
import requests
from dotenv import load_dotenv

load_dotenv()
app = Flask(__name__)

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

def guardar_medicion(data):
    url = f"{SUPABASE_URL}/rest/v1/mediciones"
    headers = {
        "apikey": SUPABASE_KEY,
        "Authorization": f"Bearer {SUPABASE_KEY}",
        "Content-Type": "application/json",
        "Prefer": "return=representation"
    }
    r = requests.post(url, headers=headers, json=data, timeout=15)
    r.raise_for_status()
    return r.json()

@app.get("/")
def inicio():
    return jsonify({"proyecto": "Backend IoT Histórico", "estado": "funcionando"})

@app.post("/mediciones")
def crear_medicion():
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "JSON requerido"}), 400

    campos = ["device_id", "tipo_variable", "valor", "unidad"]
    faltantes = [c for c in campos if c not in data]
    if faltantes:
        return jsonify({"error": "Faltan campos", "campos": faltantes}), 400

    try:
        data["valor"] = float(data["valor"])
        return jsonify({
            "mensaje": "Medición guardada correctamente",
            "data": guardar_medicion(data)
        }), 201
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.get("/mediciones")
def consultar_mediciones():
    device_id = request.args.get("device_id")
    variable = request.args.get("tipo_variable")
    limit = request.args.get("limit", "50")

    url = f"{SUPABASE_URL}/rest/v1/mediciones"
    params = {"select": "*", "order": "timestamp.desc", "limit": limit}

    if device_id:
        params["device_id"] = f"eq.{device_id}"
    if variable:
        params["tipo_variable"] = f"eq.{variable}"

    headers = {
        "apikey": SUPABASE_KEY,
        "Authorization": f"Bearer {SUPABASE_KEY}"
    }
    r = requests.get(url, headers=headers, params=params, timeout=15)
    r.raise_for_status()
    return jsonify(r.json())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

4. Código ESP32 + DHT22 en Wokwi

El ESP32 se conecta a Wokwi-GUEST y envía temperatura y humedad cada 10 segundos.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>

#define DHTPIN 15
#define DHTTYPE DHT22

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";
const char* API_URL = "https://TU-API/mediciones";

const char* DEVICE_ID = "sensor_001";
const char* UBICACION = "Quevedo, Ecuador";

DHT dht(DHTPIN, DHTTYPE);

void enviarMedicion(const char* variable, float valor, const char* unidad) {
    if (WiFi.status() != WL_CONNECTED) return;

    HTTPClient http;
    http.begin(API_URL);
    http.addHeader("Content-Type", "application/json");

    String json = "{";
    json += "\"device_id\": \"" + String(DEVICE_ID) + "\", ";
    json += "\"tipo_variable\": \"" + String(variable) + "\", ";
    json += "\"valor\": " + String(valor, 2) + ", ";
    json += "\"unidad\": \"" + String(unidad) + "\", ";
    json += "\"ubicacion\": \"" + String(UBICACION) + "\"";
    json += "}";

    int httpCode = http.POST(json);
    Serial.print(variable);
    Serial.print(" -> HTTP ");
    Serial.println(httpCode);
    Serial.println(http.getString());
    http.end();
}

void setup() {
    Serial.begin(115200);
    dht.begin();

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Conectando a WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi conectado");
    Serial.println(WiFi.localIP());
}

void loop() {
    float temperatura = dht.readTemperature();
    float humedad = dht.readHumidity();

    if (isnan(temperatura) || isnan(humedad)) {
        Serial.println("Error leyendo DHT22");
        delay(5000);
        return;
    }

    enviarMedicion("temperatura", temperatura, "C");
    enviarMedicion("humedad", humedad, "%");

    delay(10000);
}
```

5. Instalación y ejecución

1. Crear el proyecto de Supabase y ejecutar **supabase.sql**.
2. Instalar Python y crear el entorno virtual.
3. Ejecutar **pip install -r requirements.txt**.
4. Copiar **.env.example** como **.env** y completar las credenciales.
5. Ejecutar **python app.py**.
6. Crear ESP32 + DHT22 en Wokwi y colocar la URL pública de la API en **API_URL**.
7. Iniciar la simulación y verificar que aparezcan registros en Supabase.

6. Prueba

```
curl -X POST http://127.0.0.1:5000/mediciones ^  
-H "Content-Type: application/json" ^  
-d "{\"device_id\":\"sensor_001\",\"tipo_variable\":\"temperatura\",\"valor\":28.5,\"unidad\":\"C\",\"ubicacion\":\"Quevedo, Ecuador\"}"  
  
curl "http://127.0.0.1:5000/mediciones?device_id=sensor_001&limit=20"
```

El POST debe devolver HTTP 201. La consulta GET debe mostrar registros con `device_id`, `variable`, `valor`, `unidad` y `timestamp`.

7. Evidencias para completar el PDF

Captura	Evidencia
1	Circuito ESP32 + DHT22 funcionando en Wokwi.
2	Monitor serial con conexión WiFi y códigos HTTP.
3	SQL Editor de Supabase con las tablas.
4	Tabla mediciones con datos históricos.
5	Postman o navegador mostrando GET /mediciones.

8. Conclusión

El proyecto integra un dispositivo IoT simulado, comunicación HTTP, una API REST y una base de datos en la nube. Las mediciones quedan almacenadas con fecha y hora y pueden consultarse posteriormente para generar gráficos o un dashboard.

Importante: sustituir las URL y credenciales de ejemplo por las del proyecto real. La clave de Supabase debe mantenerse en el backend y no en el ESP32.